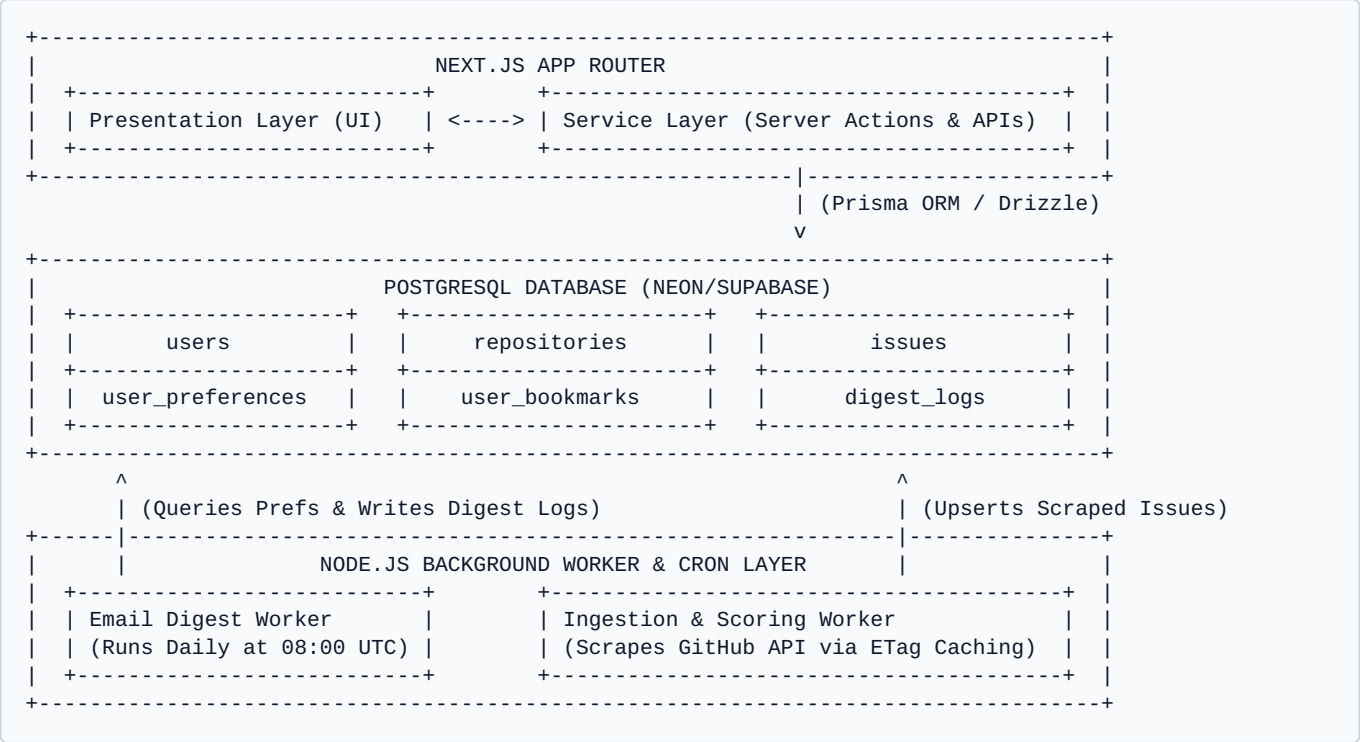
High-Level Design (HLD) & Database Architecture

Project: RepoRadar — Automated Open-Source Issue Matcher & Engagement Platform

Phase: 2 (System Architecture & ERD) **Author:** Lead Systems Architect **Database:** PostgreSQL (Neon/Supabase) via Prisma ORM

1. SYSTEM ARCHITECTURE & DATA FLOW OVERVIEW

RepoRadar is engineered as a **Modular Monolith** leveraging Next.js 15 (App Router), TypeScript, and decoupled Node.js background worker processes. The architecture enforces strict Separation of Concerns (SoC) across four distinct layers to ensure high read performance and rate-limit resilience.



2. ENTITY RELATIONSHIP DIAGRAM (ERD) & REFERENTIAL INTEGRITY

The system data model utilizes UUID primary keys, PostgreSQL native Arrays for tagging, and strict relational constraints. Below is the structural Entity Relationship representation:



3. RELATIONAL DATABASE SCHEMA (DDL SPECIFICATIONS)

3.1 Table: users

Stores core user identity authenticated via GitHub OAuth 2.0.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Internal system user ID.
`github_id`	BIGINT	UNIQUE, NOT NULL	GitHub immutable numerical user ID.
`username`	VARCHAR(255)	UNIQUE, NOT NULL	GitHub handle (e.g., torvalds).
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Primary contact email for digest alerts.
`avatar_url`	TEXT	NOT NULL	Profile picture avatar URL.
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp.

3.2 Table: user_preferences

Extension table storing 1-to-1 personalization rules, tech stack tags, and notification schedules.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`user_id`	UUID	PRIMARY KEY, REFERENCES users(id) ON DELETE CASCADE	Foreign key to parent user ID.
`preferred_tags`	TEXT[]	DEFAULT '{}'	PostgreSQL Array of target tech skills.
`min_solvability`	FLOAT	DEFAULT 50.0	Minimum score threshold (0.0 to 100.0).
`min_repo_stars`	INTEGER	DEFAULT 50	Exclude repositories below this star count.
`email_alerts`	BOOLEAN	DEFAULT true	Master toggle for automated digests.
`digest_freq`	VARCHAR(20)	DEFAULT 'DAILY'	Values: 'DAILY', 'WEEKLY', 'NEVER'.

3.3 Table: repositories

Tracks parent GitHub repositories to evaluate health metrics and store HTTP ETag headers for API rate-limit conservation.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Internal repository UUID.
`github_repo_id`	BIGINT	UNIQUE, NOT NULL	GitHub numerical repository ID.
`full_name`	VARCHAR(255)	UNIQUE, NOT NULL	Repository full path (e.g., vercel/next.js).
`stars_count`	INTEGER	DEFAULT 0	Total GitHub repository star count.
`health_score`	FLOAT	DEFAULT 50.0	Maintainer responsiveness metric (0-100).
`last_etag`	VARCHAR(255)	NULL	HTTP 304 ETag header for conditional polling.
`scraped_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of last successful cron sync.

3.4 Table: issues (Core Engine Table)

Central table storing scraped open-source issues, metadata, and computed algorithmic solvability scores.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Internal issue UUID.
`repository_id`	UUID	REFERENCES repositories(id) ON DELETE CASCADE	Parent repository link.
`github_number`	INTEGER	NOT NULL	Issue number on GitHub (e.g., #104).
`title`	TEXT	NOT NULL	Full issue title string.
`html_url`	TEXT	NOT NULL	Direct web link to GitHub issue thread.
`labels`	TEXT[]	DEFAULT '{}'	Array of labels (e.g., ['good first issue']).
`solvability_score`	FLOAT	DEFAULT 0.0	Algorithmic metric computed by background worker.
`is_open`	BOOLEAN	DEFAULT true	Boolean tracking active vs. closed state.
`comments_count`	INTEGER	DEFAULT 0	Comment count (used in score penalty w_3).
`open_prs_count`	INTEGER	DEFAULT 0	Open PR count (used in score penalty w_4).
`github_created_at`	TIMESTAMP	NOT NULL	Issue creation date (used in decay age w_1).
`is_dirty`	BOOLEAN	DEFAULT false	Flag marking score for background recalculation.

> **ARCHITECTURAL IDEMPOTENCY CONSTRAINT**
>
> A composite unique constraint is enforced on `(repository_id, github_number)`. All ingestion background workers must execute SQL `UPSERT (ON CONFLICT DO UPDATE)` operations to guarantee idempotency and eliminate duplicate issue creation during polling retries.

3.5 Tables: user_bookmarks & digest_logs

TABLE	COLUMN NAME	DATA TYPE	CONSTRAINTS & DESCRIPTION
user_bookmarks *Tracks workspace status*	`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
	`user_id`	UUID	REFERENCES users(id) ON DELETE CASCADE
	`issue_id`	UUID	REFERENCES issues(id) ON DELETE CASCADE
	`status`	VARCHAR(50)	DEFAULT 'SAVED' (Values: SAVED, INVESTIGATING, PR_OPEN, RESOLVED). Composite Unique on (user_id, issue_id).
digest_logs *Enforces FR-5.3 anti-spam*	`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
	`user_id`	UUID	REFERENCES users(id) ON DELETE CASCADE
	`issue_id`	UUID	REFERENCES issues(id) ON DELETE CASCADE
	`sent_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP. Prevents duplicate email recommendations within a 14-day window.

4. STRATEGIC DATABASE INDEXING PLAN

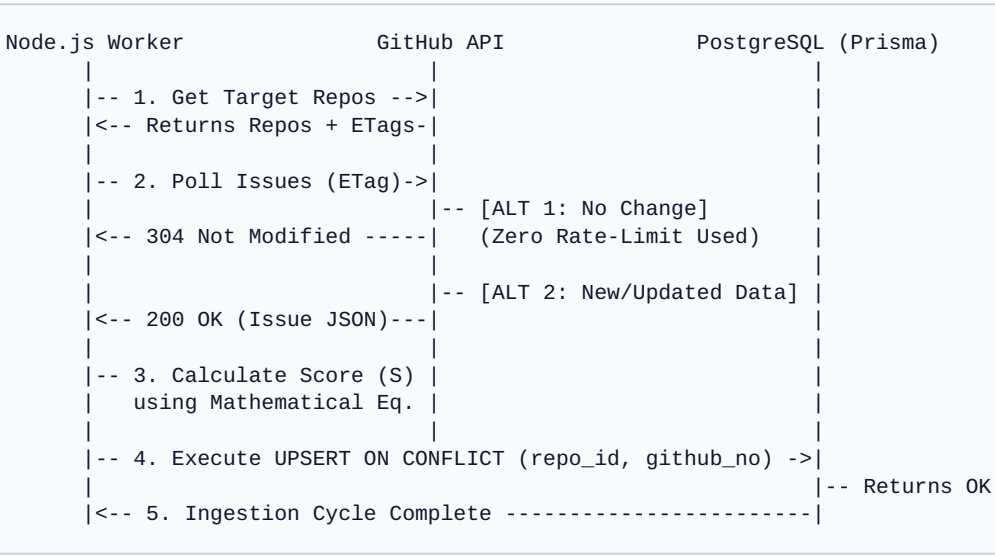
To satisfy the non-functional requirement of sub-50ms dashboard feed queries without hitting API rate limits, the following production SQL indexes are implemented:

- **GIN Index on Array Tags:** Enables high-speed array overlap queries (`&&` operator) when matching issues against user tech stacks.
  ```
  CREATE INDEX idx_issues_labels_gin ON issues USING GIN (labels);
  CREATE INDEX idx_user_prefs_tags_gin ON user_preferences USING GIN (preferred_tags);
  ```
- **Partial Composite Index for Live Dashboard Feed:** Filters active issues sorted by solvability score while ignoring closed historical data.
  ```
  CREATE INDEX idx_issues_feed ON issues (solvability_score DESC, github_created_at DESC) WHERE is_open = true;
  ```
- **Digest Anti-Spam Lookup Index:** Optimizes 14-day deduplication verification during morning email dispatches.
  ```
  CREATE INDEX idx_digest_user_issue ON digest_logs (user_id, issue_id, sent_at DESC);
  ```

5. BEHAVIORAL WORKFLOW DIAGRAMS

5.1 Sequence Diagram: Automated Ingestion & Scoring Pipeline

Illustrates how decoupled Node.js workers poll GitHub, utilize ETag conditional caching, compute solvability scores, and execute idempotent upserts into PostgreSQL.



5.2 Activity Diagram: Daily Recommendation & Email Digest Engine

Maps the execution flow of the daily 08:00 UTC notification worker enforcing dual-delivery (Web & Email) and anti-spam rules.



6. ALGORITHMIC SOLVABILITY SCORING SPECIFICATION

As established in the Phase 1 MVP criteria, every ingested open issue is processed by our weighted heuristic evaluation engine to generate a normalized solvability score (S):

$$S = \max(0, \min(100, w_1 \cdot (100 - Age_days) + w_2 \cdot (RepoHealth) - w_3 \cdot (ActiveComments) - w_4 \cdot (OpenPRs))$$

Hyperparameter Weight Allocation:

- $w_1 = 0.4$ (Freshness Decay Weight): Accounts for elapsed days since creation; older issues receive linearly decaying baseline scores.
- $w_2 = 0.6$ (Repository Health Weight): Evaluates maintainer responsiveness, merge velocity, and recent commit frequency over a 30-day window.
- $w_3 = 4.0$ (Comment Penalty): Subtracted per active comment to penalize issues with high developer competition.
- $w_4 = 40.0$ (Pull Request Roadblock Penalty): Heavy penalty applied if an open PR referencing the issue already exists, clamping unviable tasks toward 0.